

Lightweight 3×3 Depthwise Acceleration with Sliding-Window Reuse in a Tightly Coupled RISC-V CFU for INT8 MobileNetV2

line 1: 1st Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 2nd Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 3rd Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 4th Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 5th Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

line 1: 6th Given Name Surname
line 2: dept. name of
organization (of Affiliation)
line 3: name of organization (of
Affiliation)
line 4: City, Country
line 5: email address or ORCID

Abstract—[ABSTRACT PLACEHOLDER — to be written after Sections III–V are frozen ($\leq \sim 150$ words): problem, approach (3×3 depthwise CFU + sliding-window reuse), evaluation setup, verified headline results.]

Keywords—RISC-V, custom function unit, depthwise convolution, MobileNetV2, data reuse, FPGA, TinyML [PROPOSED]

I. INTRODUCTION

A. Motivation

Embedded CNN inference must fit memory budgets of kilobytes to a few megabytes under tight energy and latency limits [1]. MobileNetV2 [2] meets these limits with inverted residual blocks built from 1×1 pointwise and 3×3 depthwise convolutions. INT8 quantization allows integer-only execution in TensorFlow Lite Micro (TFLM) [3]. Even so, one inference on a soft RISC-V core takes on the order of 10⁸–10⁹ cycles [4], so hardware support is needed.

B. Problem Statement

Custom function units (CFUs) are small accelerators placed in a RISC-V pipeline and invoked through custom instructions. They offer such support at low cost [4], [5]. CFU Playground [4] adds a CFU for MobileNetV2’s dominant operator, 1×1 CONV_2D (63% of unaccelerated runtime). It reports a 55× operator speedup but only 3× on the whole model, because depthwise convolution (22.5%) and the other operators remain in software. Once the pointwise layers are accelerated, depthwise convolution dominates: in our 1×1-CFU baseline it takes about 70% of inference cycles [E1]. We therefore ask how a tightly coupled CFU can be extended to depthwise

convolution without losing the small footprint that motivates CFUs.

C. Research Gap

Prior FPGA work on MobileNet spans a range of designs. Lightweight CFUs target a single dominant operator [4]. More specialized CFU datapaths fuse the depthwise-separable block: up to 59.3× on one block with a 16,484-LUT, 173-DSP accelerator [6]. A standalone depthwise-separable accelerator uses 392 DSP slices [7]. Arithmetic-level work raises MAC density per DSP slice [7], [8]. Between these lies a less-explored point: a small depthwise unit added to an existing CFU and evaluated on the whole model. Depthwise convolution is also a different optimization problem from pointwise convolution. It has no cross-channel reuse [7], but adjacent stride-1 windows share six of nine inputs, and the fused CFU in [6] fetches and then discards every window. The end-to-end benefit and resource cost of exploiting this overlap inside a lightweight CFU are not characterized by these works.

D. Contributions

We extend the MobileNetV2 CFU of CFU Playground [4] with a dedicated 3×3 depthwise datapath and sliding-window reuse (Fig. 1). We evaluate CPU-only [E2], 1×1 CFU, 1×1 + 3×3 CFU, and 1×1 + 3×3 CFU with reuse on an Artix-7 XC7A100T at 75 MHz. We contribute:

- A **lightweight 3×3 depthwise CFU** that shares the existing CFU’s instruction interface and is invoked from the TFLM depthwise kernel. Its multipliers map to LUTs, so it adds no DSP slices or BRAM (+167 LUTs, +394 FFs at the CFU level).

- **Sliding-window reuse** for stride-1 windows that lie fully inside the input. The CFU retains the weights and six of nine inputs, and one SHIFT_RIGHT supplies each new column. This removes 71.3% of 3×3 CFU LOAD instructions [E3].
- **An end-to-end evaluation** of cycles, per-operator profiles, FPGA resources, timing, data movement, and functional correctness across all four configurations. Relative to the 1×1 CFU, the 3×3 CFU reduces inference cycles by 25.8%, and 30.8% with reuse, and all designs meet timing [E1], [E2].

[FIGURE 1 PLACEHOLDER — single-column, ≈ 2.0 in]

Fig. 1. Overall system architecture: TFLM on VexRiscv, CFU with the existing 1×1 path and the new 3×3 depthwise unit, Artix-7 SoC. [PLACEHOLDER CAPTION]

II. BACKGROUND AND RELATED WORK

A. MobileNetV2 and Embedded CNN Inference

A MobileNetV2 inverted residual block applies a 1×1 expansion, a 3×3 depthwise convolution, and a 1×1 projection [2] (Fig. 2). The 1×1 layers are per-pixel matrix–vector products that reuse each weight across all pixels and each input across all output channels, so they map well onto MAC arrays [4]. Depthwise convolution filters each channel with its own 3×3 kernel. It needs far fewer operations, but inputs cannot be reused across channels [7]. Its only reuse is spatial, between overlapping windows of one channel. Under INT8 quantization, both layer types run as integer kernels with per-channel requantization [3].

[FIGURE 2 PLACEHOLDER — single-column, ≈ 1.4 in]

Fig. 2. MobileNetV2 inverted residual block and operator mapping: 1×1 expansion/projection → existing 1×1 CFU; 3×3 depthwise → new 3×3 CFU; other operators → software. [PLACEHOLDER CAPTION]

B. TensorFlow Lite Micro and RISC-V CFU-Based TinyML

TFLM executes a quantized model by calling one kernel per operator [3]. CFU Playground [4], [5] pairs TFLM with a

LiteX SoC built around the VexRiscv soft core on an FPGA. A CFU sits in the CPU pipeline and executes R-type custom instructions with two source operands and one result. Larger operations use instruction sequences and CFU-internal state [5]. Acceleration is introduced per operator: a specialized kernel issues CFU instructions when the layer parameters allow it, and all other operators stay in software. Combined with cycle-level profiling, this supports co-design one bottleneck at a time [4].

C. RISC-V CFU Acceleration for MobileNetV2

In the CFU Playground MobileNetV2 study, about 95% of unaccelerated cycles fall in 1×1 convolution (63%), depthwise convolution (22.5%), and 3×3 convolution (11%) [4]. Its CFU buffers filters and inputs, performs four-way INT8 MACs, and requantizes in hardware. This gives 55× on 1×1 CONV_2D but 3× on the whole model [4]. In a separate keyword-spotting study on a smaller FPGA, the authors note that depthwise convolution has a different memory access pattern. A separate depthwise CFU did not fit there [4]. Yildirim and Ozturk [6] remove intermediate feature-map buffers with a fused, deeply pipelined CFU. It reaches 59.3× on one bottleneck block, but the accelerator alone uses 16,484 LUTs and 173 DSPs, against 4,438 LUTs for the whole baseline SoC. Sabih *et al.* [9] extend CFU Playground for sparse, pruned DNNs. We keep the dense INT8 model and the CFU Playground execution model, and extend its operator coverage.

D. Depthwise-Convolution and Data-Reuse Optimization

At stride 1, consecutive 3×3 windows in a row overlap in two columns and share six of nine inputs. A CFU that receives each window independently reloads this overlap. The fused CFU of [6] uses exactly such a no-local-reuse depthwise dataflow. Dedicated accelerators exploit the overlap by spatial tiling: [7] reads a 9×9 tile to produce 7×7 depthwise outputs. Because depthwise convolution lacks cross-channel reuse [7], spatial reuse is its main lever on data movement, which increasingly limits ML hardware performance and energy [10]. DSP packing [7] and mixed DSP/LUT MAC units [8] instead raise arithmetic density. They are orthogonal to reuse and are not our focus.

E. Positioning of This Work

CFU Playground [4] is the closest system, and we reuse its 1×1 CFU and execution model unchanged. More specialized, fused CFU datapaths [6] and standalone accelerators [7] gain more per layer at much higher cost. The accelerator in [6] uses about 6× the LUTs and over 40× the DSP slices of our complete CFU. Arithmetic-level techniques [7], [8] and sparsity [9] are complementary. This work takes the intermediate point. It adds a small LUT-based 3×3 depthwise unit beside the 1×1 CFU, uses sliding-window reuse to exploit the intra-channel overlap that [6] leaves unused, and evaluates dense INT8 MobileNetV2 end to end, together with resources, timing, and correctness. Section III presents the architecture.

III. PROPOSED SYSTEM AND CFU ARCHITECTURE

[SECTION III PLACEHOLDER — not yet drafted. Planned: SoC/TFLM integration; existing 1×1 CFU; 3×3 depthwise CFU (CONFIGURE, LOAD, RUN, GET_RESULT);

LUT-based MACs; 3-stage adder pipeline); sliding-window reuse (SHIFT_RIGHT, retained window/weight state, eligibility). Source: ARCHITECTURE_NOTES.md.]

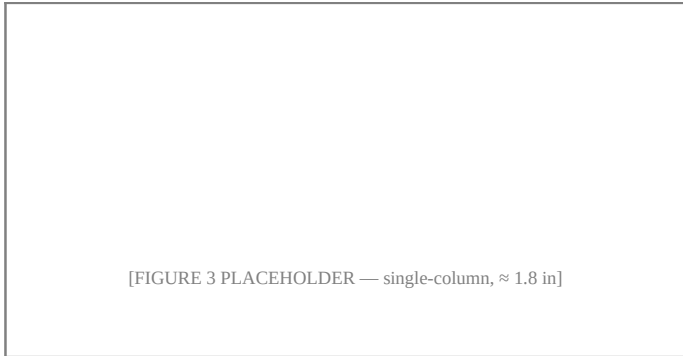


Fig. 3. 3×3 depthwise CFU datapath and sliding-window reuse: window A B C/D E F/G H I → B C J/E F K/H I L, six inputs retained, one new column per SHIFT_RIGHT. [PLACEHOLDER CAPTION]

IV. EXPERIMENTAL METHODOLOGY

[SECTION IV PLACEHOLDER — not yet drafted. Planned: platform (Nexys4 DDR, XC7A100T, 75 MHz, Vivado 2024.1), model and input, four configurations, metrics (cycles, profiler, post-place utilization, post-route timing, instruction counts, functional tests). Pending: CPU-only evidence [E2], run platform/commit [E1].]

V. RESULTS AND DISCUSSION

[SECTION V PLACEHOLDER — not yet drafted. Source: evidence/tables/ and figures/graphs/.]

VI. LIMITATIONS AND FUTURE WORK

[SECTION VI PLACEHOLDER — not yet drafted.]

VII. CONCLUSION

[SECTION VII PLACEHOLDER — not yet drafted.]

REFERENCES

- [1] C. Banbury *et al.*, “MLPerf Tiny benchmark,” 2021, arXiv:2106.07597.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018. [VERIFY: PDF not in literature/]
- [3] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” *Proc. Mach. Learn. Syst.*, vol. 3, pp. 800–811, 2021. [VERIFY: PDF not in literature/]
- [4] S. Prakash *et al.*, “CFU Playground: Full-stack open-source framework for tiny machine learning (TinyML) acceleration on FPGAs,” 2023, arXiv:2201.01863v3. [CHECK: published version]
- [5] S. Prakash *et al.*, “CFU Playground: Want a faster ML processor? Do it yourself!,” in *Proc. Design, Automation & Test in Europe Conf. (DATE)*, 2023.
- [6] M. Yildirim and O. Ozturk, “RISC-V based TinyML accelerator for depthwise separable convolutions in edge AI,” 2025, arXiv:2511.21232.
- [7] X. Li, H. Huang, Y. Liu, X. Hu, and X. Xiong, “A digital signal processor-efficient accelerator for depthwise separable convolution,” *Electron. Lett.*, vol. 58, no. 7, pp. 271–273, Mar. 2022, doi: 10.1049/ell2.12435.
- [8] M. Véstias, R. Duarte, J. T. de Sousa, and H. Neto, “Parallel dot-products for deep learning on FPGA.” [VERIFY: venue and year not in PDF]
- [9] M. Sabih, A. Karim, J. Wittmann, F. Hannig, and J. Teich, “Hardware/software co-design of RISC-V extensions for accelerating sparse DNNs on FPGAs,” in *Proc. Int. Conf. Field-Programmable Technol. (FPT)*, 2024.
- [10] M. S. Vahdatpour and Y. Zhang, “Energy-efficient software–hardware co-design for machine learning: From TinyML to large language models,” 2026, arXiv:2603.23668.